

Programming Fundamentals

C-CS112 Spring 2022

LECTURE 04: Loops in



جامعة مصر للمعلوماتية
EGYPT UNIVERSITY
OF INFORMATICS



جامعة مصر للمعلوماتية
EGYPT UNIVERSITY
OF INFORMATICS

Lecture Topics

- Increment and Decrement Operators
- for/while comparison
- The Empty Statement
- The break-Statement
- Nested Loops

Increment and Decrement Operators

- **Increment operator** (++)
 - Can be used instead of: `+= 1`
- **Decrement operator** (--)
 - Can be used instead of: `-= 1`
- **Preincrement**
 - Operator is used before the variable (`++c` or `--c`)
 - Variable is **changed before the expression** it is in is **evaluated**
- **Postincrement**
 - Operator is used after the variable (`c++` or `c--`)
 - **Expression executes before the variable is changed**

Increment and Decrement Operators

```
#include <iostream>
using namespace std;

int main()
{
    int n = 1, m = 7;
    n++;
    cout << "n = " << n << endl;
    m--;
    cout << "m = " << m << endl;
}
```



What will be
the output?



Increment and Decrement Operators

Operator	Sample expression	Explanation
++	++a	Increment a by 1, then use the new value of a in the expression in which a resides.
++	a++	Use the current value of a in the expression in which a resides, then increment a by 1.
--	--b	Decrement b by 1, then use the new value of b in the expression in which b resides.
--	b--	Use the current value of b in the expression in which b resides, then decrement b by 1.

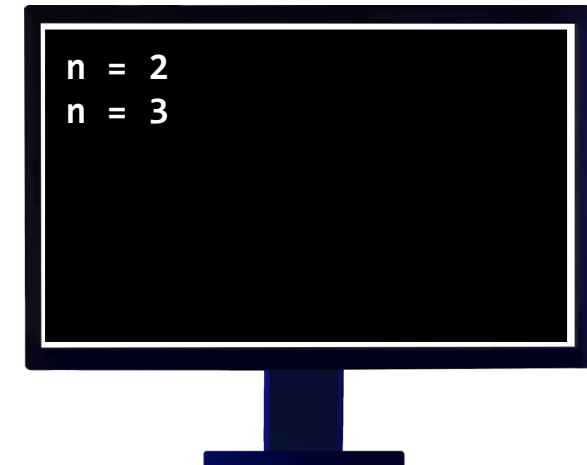
Increment and Decrement Operators

```
#include <iostream>
using namespace std;

int main()
{
    int n = 1;
    n++;
    cout << "n = " << n << endl;
    ++n;
    cout << "n = " << n << endl;
}
```



What will be
the output?

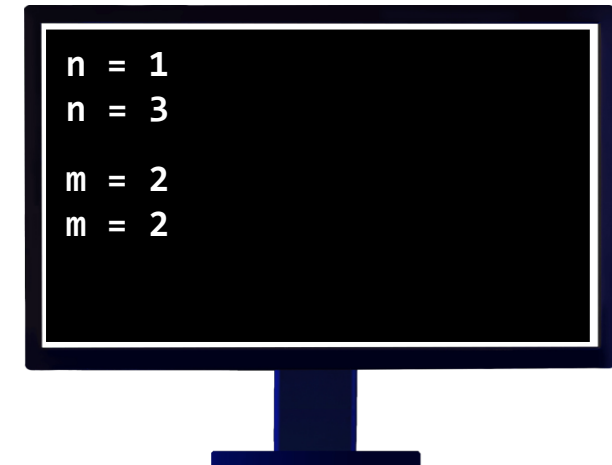


Increment and Decrement Operators

```
#include <iostream>
using namespace std;

int main()
{
    int n = 1;
    cout << "n = " << n++ << endl;
    cout << "n = " << ++n << endl;

    int m = 1;
    cout << "m = " << ++m << endl;
    cout << "m = " << m++ << endl;
}
```



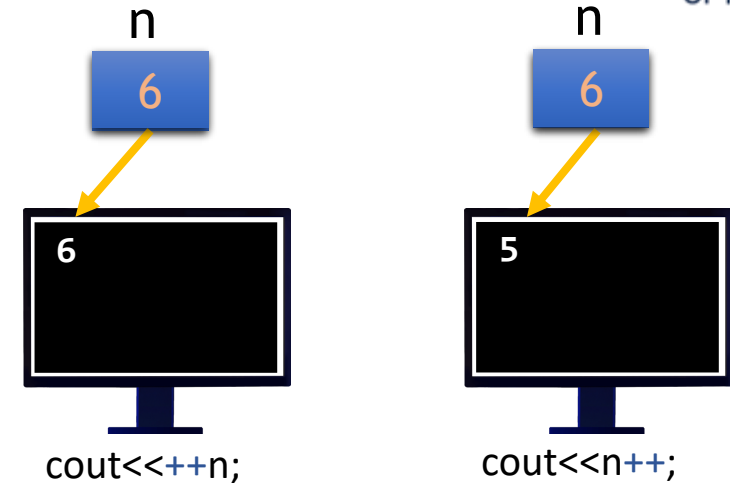
Increment and Decrement Operators

- If n equals 5, then :

`cout << ++n;`

`cout << n++;`

- In either case, n now has the value of 6



- When the variable not in an expression, pre-incrementing and post-incrementing operators have the **same effect**

- Example:

```
++n;
cout << n;
```

has the same effect as

```
n++;
cout << n;
```

Increment and Decrement Operators

- The increment operator (**++**) is used to **increase** the value of a variable by one.
 - statements such as: `number++;`
- The decrement operator (**--**) is used to **decrease** the value of a variable by one.
 - statements such as: `number--;`
- The increment and decrement operators can also be used in expressions:

```
int number = 2;  
int value_incremented = number++;  
cout<< value_incremented<<endl;  
int value_decremented = number--;  
cout<< value_decremented<<endl;
```

 - What is the output?

Increment and Decrement Operators

- What is the difference if the expressions are changed to be

```
int number = 2;  
int value_incremented = ++number;  
cout<< value_incremented<<endl;  
int value_decremented = --number;  
cout<< value_decremented<<endl;
```

for/while Loop Comparison

- Two programs that add the numbers from 1 to 10. Can you find another way?

while loop

```
int sum = 0;
int n = 1;

// add the numbers 1 - 10
while (n <= 10) {
    sum = sum + n;
    n++;
}
```

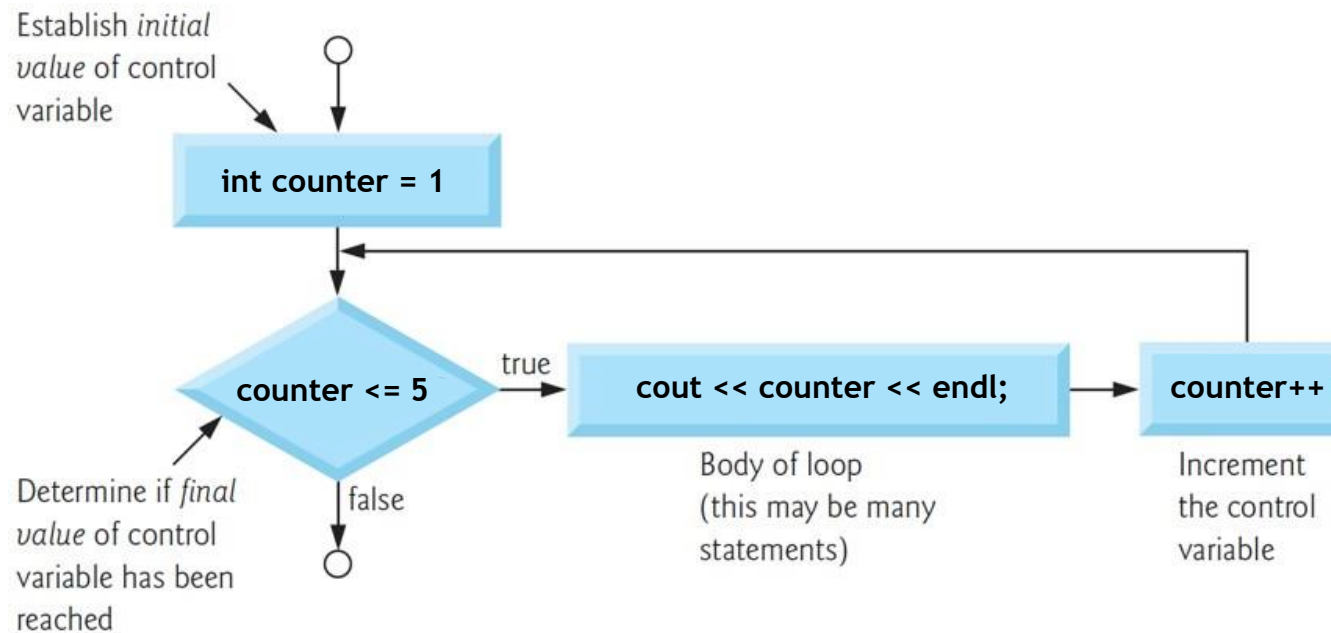
for loop

```
int sum = 0;

// add the numbers 1 - 10
for (int n = 1; n <= 10; n++)
{
    sum = sum + n;
}
```

for Statement Flowchart

```
for (int counter = 1; counter <= 5; counter++)  
    cout << counter << endl;
```



for Statement Trace

- Example: `for (int num = 1; num < 4; num++)`

Iteration	Control Variable	Condition	
1 st	num = 1 2	is 1 < 4 ?	✓ YES
2 nd	num = 2 3	is 2 < 4 ?	✓ YES
3 rd	num = 3 4	is 3 < 4 ?	✓ YES
4 th	num = 4	is 4 < 4 ?	✗ NO STOP

for Statement Example2

- Calculate the square of each number in a range (1 through 5)

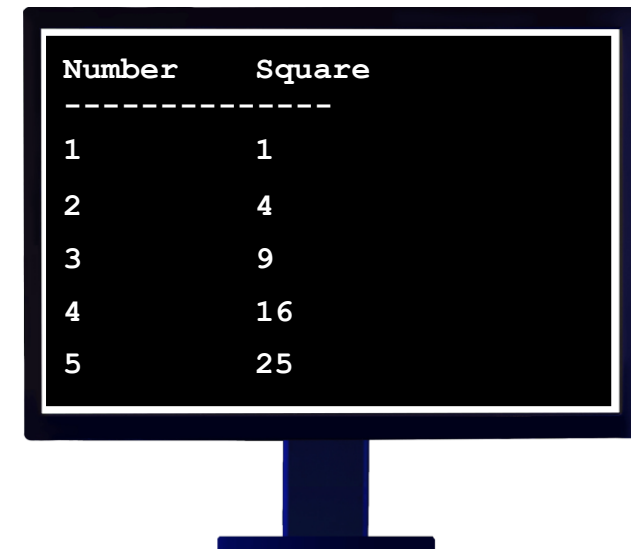
```
#include <iostream>
using namespace std;

int main()
{
    // Print the table headings.
    cout << "Number\tSquare\n";
    cout << "-----\n";

    for (int num = 1; num <= 5; num++) {
        cout << num << "\t" << num * num << endl;
    }
}
```



How can we display numbers in descending order instead?



Number	Square
1	1
2	4
3	9
4	16
5	25

for-loop Details

- Initialization and update actions of for-loops often contain more complex expressions
 - Here are some samples

```
for(n = 0 ; n > -100 ; n = n -7)
```

```
for(double x = pow(y,3.0); x > 2.0; x = sqrt(x) )
```

← #include <cmath>

```
for (int i = 0, j = 10; i != j; i++, j--)
```

The Empty Statement

- A **semicolon** creates a **C++ statement**
 - Placing a semicolon after `x++` creates the statement
`x++;`
 - Placing a semicolon after nothing creates an **empty statement** that compiles but does nothing

```
cout << "Hello" << endl;  
;  
cout << "Good Bye"<< endl;
```

Extra Semicolon

- Placing a semicolon after the **parentheses of a for loop** creates an empty statement as the **body of the loop**

- Example:

```
for(int count = 1; count <= 10; count++);  
    cout << "Hello";
```

prints **one** "Hello", but not as part of the loop!

- The empty statement is the body of the loop
- `cout << "Hello";` is not part of the loop body!

Which Loop To Use?

- Choose the **type of loop** late in the design process
 - First design the loop using pseudocode
 - Translate the pseudocode into C++
 - The translation generally makes the choice of an appropriate loop clear
 - **while-loops** are used for all other loops when there might be occasions when the **loop should not run**
 - **do-while loops** are used for all other loops when the loop **must always run at least once**
 - **for-loops** are typically selected when doing **numeric calculations**, especially when using a variable changed by equal amounts each time the loop iterates

The break-Statement

- There are times to **exit a loop** before it **ends**
 - If the loop checks for invalid input that would ruin a calculation, it is often best to end the loop
- The **break-statement** can be used to exit a loop **before normal termination**
 - Be careful with nested loops!
 - Using break **only exits the loop in which the break-statement occurs**

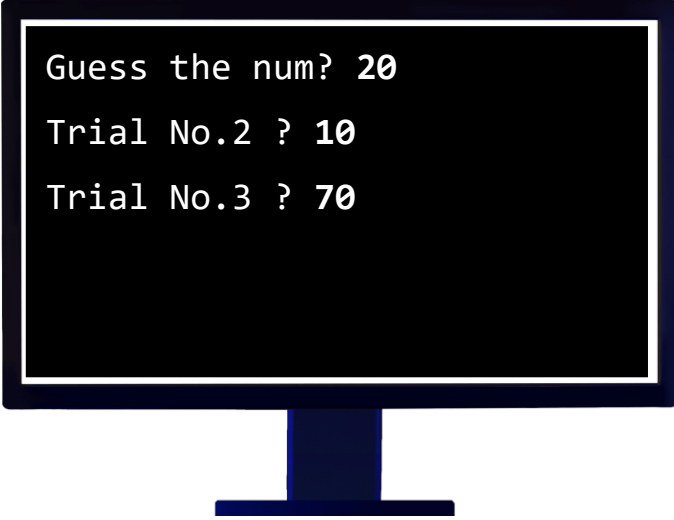
The break-Statement: Example

```
#include <iostream>
using namespace std;

int main(){
    int num = 50, guess;
    cout << "Guess the num? ";
    int count = 1;
    while (count++ < 4) {
        cin >> guess;
        if (guess == num) {
            cout << "Good Job!" << endl;
            break;
        }
        if (count != 4)
            cout << "Trial No." << count << " ? ";
    }
}
```



What will be the output if count++ is replaced by ++count?



```
Guess the num? 20
Trial No.2 ? 10
Trial No.3 ? 70
```

Nested Loops

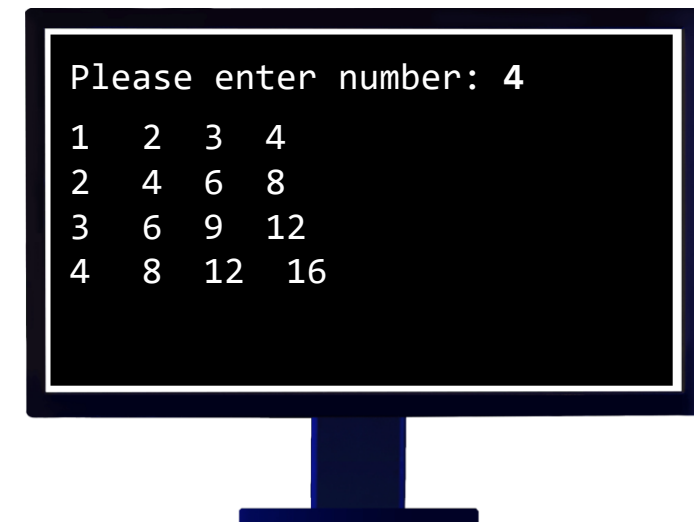
- The body of a loop may contain any kind of statement, including **another loop**
 - When loops are nested, all iterations of the inner loop are executed for each iteration of the outer loop
 - Give serious consideration to making the inner loop a function call to make it easier to read your program

Nested Loops: Example

- A program that print out all the values for a $n \times n$ multiplication table.

```
#include <iostream>
using namespace std;

int main(){
    int n;
    cout << "Please enter number: ";
    cin >> n;
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n; j++)
            cout << i * j << " ";
        cout << endl;
    }
}
```



Debugging Example

- The following code is supposed to conclude with the variable product containing the product of the numbers 2 through 5.

```
#include <iostream>
using namespace std;

int main(){
    int next = 2, product = 1;
    while (next < 5) {
        next++;
        product = product * next;
    }
}
```



Does the code contain any logical errors (bugs)?

Exercise 1

(Product of a series) Write a program to calculate the product of the following series:

$$\frac{1}{5} \times \frac{1}{9} \times \frac{1}{13} \times \frac{1}{17} \times \frac{1}{21} \times \frac{1}{25} \times \dots \times \frac{1}{93} \times \frac{1}{97}$$

Exercise 2

(Find the lowest price) Write a program that prompts the user to enter the number of items and each item's name and price, and finally displays the name and price of the item with the lowest price.

Exercise 3

(Find numbers divisible by 3 and 4) Write a program that displays all the numbers from 1 to 500, 5 per line, that are divisible by 3 and 4. Numbers are separated by exactly one space.

Exercise 4

Write a program that asks the user to enter today's sales for five stores. The program should then display a bar graph comparing each store's sales. Create each bar in the bar graph by displaying a row of asterisks. Each asterisk should represent \$100 of sales.

Here is an example of the program's output.

```
Enter today' s sales for store 1: 1000 [Enter]
Enter today' s sales for store 2: 1200 [Enter]
Enter today' s sales for store 3: 1800 [Enter]
Enter today' s sales for store 4: 800 [Enter]
Enter today' s sales for store 5: 1900 [Enter]
```

```
SALES BAR CHART
( Each * = $100)
Store 1: *****
Store 2: *****
Store 3: *****
Store 4: *****
Store 5: *****
```



جامعة مصر للمعلوماتية
EGYPT UNIVERSITY
OF INFORMATICS

End of Lecture!

Thanks for your Attention!



جامعة مصر للمعلوماتية
EGYPT UNIVERSITY
OF INFORMATICS